

Early Diagnosis of Wasted Computation in Multi-Agent LLM Systems via Failure-Aware Observability

Xianyou Li

New York University
New York, NY 10012, USA
xl4230@nyu.edu

Weiran Yan

Independent Researcher
Santa Clara, CA 95051, USA
yanwr2016@gmail.com

Yichao Wu

Northeastern University
Boston, MA 02115-5000, USA
wu.yicha@northeastern.edu

Penghao Liang

Northeastern University
Boston, MA 02115-5000, USA
liang.p@northeastern.edu

Mengwei Yuan

Independent Researcher
Milpitas, CA 95035-7246, USA
yuanmw1998@gmail.com

Jianan Liu

Independent Researcher
Austin, TX 78613-5377, USA
jiananliu2408@gmail.com

Jing Yang

Washington University in St. Louis
St. Louis, MO 63130, USA
jing.y@wustl.edu

Abstract—Failure-aware observability diagnoses wasted computation in multi-agent LLM systems before final-answer evaluation can explain what went wrong. We propose a trace-based framework for a three-agent architecture—orchestrator, search agent, and execution agent—that converts structured events into online signals for loops, budget pressure, low information gain, and tool instability, then adds offline semantic grounding metrics and selective LLM-as-judge evaluation. On 165 GAIA validation traces under identical caps, 98 runs produce usable final answers and 67 fail or stop without one. Among warned failed runs, 58.1% of tokens are spent after the first warning on average, indicating substantial opportunity for intervention. A 10-task Level-2 pilot uses warnings to diversify search or require evidence, reducing post-warning token fraction from 0.638 in the baseline to 0.304. The results support a layered design: cheap online signals help the orchestrator redirect or halt redundant behavior, while deeper semantic checks identify whether completed answers are grounded enough to trust.

Keywords—MULTI-AGENT SYSTEMS, TRAJECTORY ANALYSIS, EVALUATION, LLM-AS-JUDGE, WASTED COMPUTATION

I. INTRODUCTION

Tool-using and multi-agent LLM systems interleave reasoning, retrieval, and action [1]–[3]. This design can increase task coverage, but it also creates long trajectories whose failures are hard to interpret from final answers alone. Benchmarks such as GAIA [6] measure final task success, yet a final score compresses many different processes into one endpoint: redundant searches, unproductive retrieval, execution errors, weak evidence, repeated coordination decisions, or no final answer.

Recent work makes this diagnostic gap explicit. Faulty-agent analysis studies how collaboration resilience depends on system structure [4], while MAST provides a failure taxonomy, annotated multi-agent traces, and an LLM-as-judge annotation pipeline [5]. Related trajectory-centered work models agent paths with large-and-small model collaboration

[7]. Distributed-system tracing connects observed symptoms to internal execution paths [8], but multi-agent LLM systems need a related form of observability: their costs arise from model calls, retrieval, tools, and execution, while their failures can be semantic rather than exceptional. Evaluation methods are also shifting. LLM-as-judge methods provide semantic assessment but add model-call cost; sentence embeddings offer cheaper semantic checks [9]. Industry guidance likewise treats transcript records, graders, and tracked metrics as core components of agent evaluation [10].

This paper treats wasted computation as an observability problem. We define wasted computation as tokens, tool calls, computation attempts, and latency spent on runs that do not produce a usable, grounded final answer. We use cost-correlated proxies available during execution rather than direct hardware energy or provider billing. Since agentic and test-time-scaling workflows can increase inference energy demand [12], the central claim is that multi-agent LLM systems require failure-aware observability in addition to final-answer evaluation.

We instantiate this claim in a three-agent question-answering system composed of an orchestrator, a search agent, and an execution agent. Runs are stored as structured event traces. Online signals identify repeated actions, low information gain, missing evidence, and tool-failure streaks; offline analysis derives reliability, grounding, semantic-support, and budget metrics from the same records. Related systems improve agent fallback, memory, retrieval, root-cause investigation, and iterative decision loops [13]–[15]; broader evaluation work shows that task labels can shift across platforms [16]. Our focus is orthogonal: trace-level signals that explain how existing runs spend computation and where failure becomes visible.

The contributions of this paper are:

- A failure taxonomy distinguishing execution failure, orchestration loops, evidence failure, no-final termination, and cost waste.

- A trace-based observability framework mapping failure modes to online metrics, offline semantic evaluations, and warning-triggered interventions.
- A level-stratified GAIA analysis showing how warnings, grounding signals, and budget proxies relate to failed computation.

II. SYSTEM AND TRACE RECORDS

A. Agent Architecture

The evaluated system has three roles: an orchestrator, a search agent, and an execution agent. The orchestrator chooses among five structured actions: seeking information, inspecting sources, using task materials, invoking bounded execution, or producing a final response. The search agent returns source summaries, while the execution agent processes task materials under controlled runtime limits. All reported runs use deterministic, low-cost model settings.

Figure 1 separates the action interface from the observability layer. The action space changes the run state; the observability dimensions summarize what those actions produce. Seek and inspect actions feed information-change and evidence signals, material and execution actions feed budget and tool signals, and repeated normalized action signatures expose loops across action types.

B. Structured Trace Records

Each run is represented as an ordered event record. Events contain a run identifier, timestamp, event type, agent role, and structured payload, allowing online warnings and offline metrics to be reconstructed from the trace itself. The persistent storage backend is an implementation detail.

Table I summarizes the mapping from failure modes to observable signals. It is read row-wise: each row names an operational symptom, the signal layer, measurable trace features, and the corresponding triage interpretation. These signals are not automatic stop rules; they indicate when to trigger deeper evaluation, a behavioral nudge, or post-run analysis.

III. FAILURE-AWARE METRICS

Let a run be r and let E_r denote its ordered event trace. We compute metrics at two levels: online metrics emitted during the run and offline metrics computed from the completed trace.

A. Execution and Tool Reliability

Tool error rate measures the fraction of tool results whose status is `error`:

$$\text{ToolErr}(r) = \frac{N_{\text{err}}(r)}{N_{\text{tool}}(r)}. \quad (1)$$

Here N_{tool} is the number of tool-result events in the run and N_{err} is the subset with error status. We separately measure execution success and retry outcomes to distinguish unreliable tools from ineffective recovery behavior.

B. Orchestration and Loop Signals

Orchestration signals characterize whether the agentic process is still making progress. The system represents each action by a normalized signature and uses repeated signatures or alternating repeated signatures as loop proxies. It also records coarse information-gain signals, such as new source references or newly extracted evidence. Repeated low-gain steps are treated as stagnation indicators rather than as direct correctness judgments.

C. Evidence and Grounding Signals

Grounding metrics estimate whether a final answer is supported by the trace. Always-on signals record evidence availability and citation consistency. Deeper offline analysis estimates answer-evidence alignment with a lightweight semantic encoder. The reported artifact uses a local LSA-style representation: term-frequency vectors are projected into a lower-dimensional semantic space, providing a cheap approximation of sentence relatedness without an additional model call. The same interface can support Sentence-BERT-style embeddings [9]. The sentence-level support proxy is:

$$\text{Support}_\tau(r) = \frac{1}{|S_a|} \sum_{s \in S_a} \mathbb{I} \left[\max_{c \in C_e} \cos(f(s), f(c)) \geq \tau \right], \quad (2)$$

where S_a are answer sentences, C_e are evidence chunks, f is the embedding function, and $\tau = 0.65$ in the reported analysis.

D. Selective LLM-as-Judge Evaluation

We use an LLM judge only as an offline grounding evaluation. For each evaluated trace, the judge receives the original question, the final answer, and the recorded evidence. A strict prompt asks whether the evidence supports the answer and returns a support score in $[0, 1]$. This evaluation measures grounding, not benchmark correctness. The judge does not use prior knowledge to answer the task and does not control the agent trajectory. We use deterministic decoding and cache responses to avoid repeated calls.

E. Wasted Computation Proxy

For a configurable cost vector, run cost can be approximated as

$$C_r = \alpha T_r + \beta H_r + \gamma R_r + \delta X_r, \quad (3)$$

where r indexes a run, T_r is total tokens, H_r is tool calls, R_r is retries, X_r is execution attempts, and $\alpha, \beta, \gamma, \delta$ are deployment-specific unit-cost coefficients for these resources. These coefficients convert heterogeneous resource counts into one comparable cost scale; for example, a deployment could set them from provider token prices, tool fees, measured latency, or energy accounting. A wasted-computation ratio can then be defined as

$$\text{WCR} = \frac{\sum_r C_r \cdot \mathbb{I}[\text{unusable}(r)]}{\sum_r C_r}. \quad (4)$$

Here $\mathbb{I}[\text{unusable}(r)]$ is an indicator that equals 1 when run r terminates without a usable final result and 0 otherwise. In

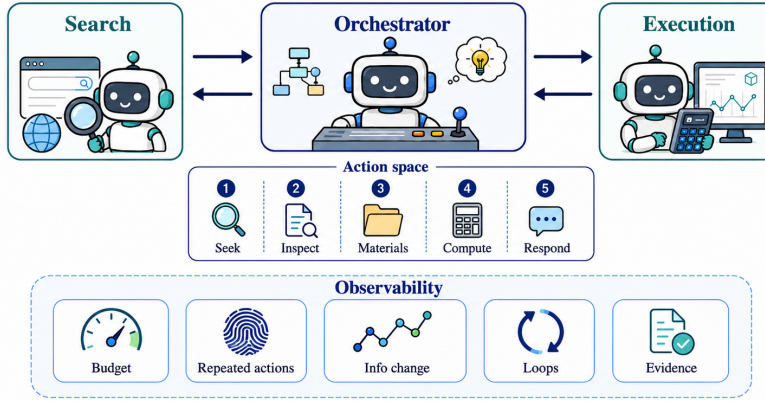


Fig. 1. Agent architecture and observability signals. The orchestrator coordinates the search and execution agents through a five-action interface. The observability layer summarizes the resulting trace along budget, repeated action, information-change, loop, and evidence dimensions.

TABLE I
FAILURE-AWARE OBSERVABILITY MAP. METRICS ARE GROUPED BY EXPOSED FAILURE MODE AND SIGNAL LAYER.

Failure mode	Layer	Observable signal	Operational interpretation
Tool instability	Search / execution	Tool error rate, retry summaries, latency	Calls are consuming budget without returning usable state.
Execution failure	Execution	Execution success rate, setup/timeout classes	External execution is not a reliable recovery path.
Repeated action loop	Orchestration	Repeated action keys, ABAB cycle labels, cache hits	The run is spending computation without changing strategy.
Low information gain	Orchestration	New URLs, extracted fact count, low-gain streaks	Search or page opening is no longer adding task-relevant state.
Evidence failure	Grounding	Evidence-present rate, citation consistency, answer-evidence similarity, sentence support	The final answer may be unsupported even if the run terminates.
Budget waste	System	Tokens, tool calls, budget pressure, post-warning remaining budget	Computation after an early warning is available for intervention or evaluation.

the current experiment, dollar-cost coefficients are left unset because provider pricing, hardware allocation, caching, and tool billing vary across deployments. Setting one artificial dollar vector would imply a billing model that the experiment does not measure. We therefore report token and tool-call counts directly and treat them as auditable computation-cost proxies rather than as monetary or energy measurements.

IV. EXPERIMENTAL SETUP

We evaluate on the complete local GAIA 2023 validation snapshot: 165 tasks split across benchmark-provided difficulty levels 1, 2, and 3 as 53/86/26 tasks. We treat the levels as increasing difficulty strata rather than as separate datasets: level 1 contains the simplest tasks, level 2 is the largest intermediate stratum, and level 3 is the smallest high-difficulty stratum in the local snapshot. The final runs were configured with identical caps: 8 orchestrator steps, 8 orchestrator-paid tool calls, 30-second model-request timeouts, deterministic decoding, and a 25,000-token soft ceiling. Trace metadata records the task level, task identifier, model settings, budget limits, timeout, batch name, and source-code fingerprint; all reported level batches use the same recorded caps.

We define a usable final result operationally: the system must produce a final response marked usable by the orchestrator.

Failed or no-final outcomes include explicit failed finals and traces without a final response. This definition does not claim benchmark correctness; grounding quality and correctness are evaluated separately.

We use two capped offline evaluations with gpt-4.1-mini, a model in the GPT-4.1 family [11]. A 10-trace grounding evaluation compares final answers with recorded evidence. A 30-trace correctness evaluation samples usable finals across levels and compares them with the GAIA reference answers. Both evaluations are cached and capped so that LLM-as-judge evaluation remains a selective validation layer rather than an always-on metric.

V. RESULTS

Figure 2 summarizes the complete validation-snapshot run. On correctness, we report a selective offline evaluation over 30 runs that produced usable finals (10 per level). Using gpt-4.1-mini as the grader against GAIA validation reference answers, 12/30 evaluated usable finals are judged correct (level-1: 6/10; level-2: 4/10; level-3: 2/10). We treat this as a sanity check that separates “usable final produced” from benchmark correctness; it is not a full accuracy estimate over all tasks or over failed runs.

To quantify wasted computation after early warnings, we measure the fraction of tokens consumed after the first triage label in failed runs. Across the 67 failed runs, 56 emitted at least one warning; among those warned failed runs, the mean post-warning token fraction is 0.581 and the median is 0.611.

A. Cross-Level Run Behavior

Table II is a workload-level summary. *Usable* and *Failed* are operational counts; the other columns are level means over all runs. This reports the computation and signal prevalence seen online, while later subsections analyze failure mechanisms and post-warning waste on failed runs and grounding/correctness on final-bearing runs. *Tools* is mean orchestrator-paid tool calls; *Support* is Eq. (2) with $\tau = 0.65$.

Evidence availability is high in levels 2 and 3, but average support does not increase monotonically because failed/no-final traces contribute little or no answer support. Evidence presence, operational completion, and grounded correctness should therefore be treated as related but separate outcomes. Citation gaps remain visible: evidence is present without citations in 28.3%, 36.0%, and 46.2% of levels 1–3.

B. Failure Signals

Trace labels separate unusable outcomes by mechanism. Level-1 failures include insufficient evidence, loops, and step-limit termination; level-2 and level-3 failures are dominated by loops and step limits. Loop warnings appear in 16/31/9 runs across levels 1–3. Lower-frequency labels expose low information gain, tool-failure streaks, evidence gaps, execution without usable output, and blocked near-duplicate search.

C. Wasted Computation After Warning

For each run, we compare the token budget snapshot at the first triage label with the final token count. Among warned failed runs, 54.5%, 63.8%, and 46.8% of tokens in levels 1–3 are spent after the first warning on average, showing that warnings often arrive early enough for budget-aware intervention.

D. Intervention Pilot

We evaluate a minimal warning-triggered policy on 10 level-2 tasks, chosen because level 2 is the largest stratum and has many loop warnings. The policy diversifies information seeking after loop warnings and requires opened evidence after evidence-gap warnings. The pilot produces 8/10 usable finals; the two failed runs end by step limit and repeated loop. Among warned failures, the mean post-warning token fraction is 0.304 versus 0.638 in the full level-2 baseline. The comparison is directional because the pilot has one warned failure, but it shows how warnings can trigger low-cost behavioral nudges.

E. Trace Case Studies

Aggregates show correlation; trace excerpts show how the signals arise. Table III compresses representative level-1–3 traces to the decision-relevant sequence, exposed signal, and triage implication. The cases generalize four patterns: repeated actions can waste budget despite successful tools; usable finals

can be weakly grounded; tool errors can recover when later evidence is strong; and high-level tasks can keep spending tokens after multiple warnings without reaching a final answer.

F. Metric-Layer Comparison

Figure 3 reports Pearson r for all 165 runs (98 usable, 67 unusable) and the 10-trace judge subset. Since usable and unusable are complementary outcomes, their signs mirror. For unusable outcomes, the largest cheap-signal association is any triage label ($r = 0.550$), followed by tool-error rate ($r = 0.391$), total tokens ($r = 0.267$), and tool calls ($r = 0.176$). Semantic support and answer–evidence similarity associate with usable outcomes ($r = 0.594, 0.568$) and with judge support ($r = 0.431, 0.379$). These are descriptive correlations, not causal or accuracy estimates.

VI. DISCUSSION, LIMITATIONS, AND CONCLUSION

The full run supports a layered observability design. Cheap trace signals expose tool instability and stalled trajectories during execution; semantic metrics separate evidence presence from answer support; selective LLM-as-judge evaluation validates those proxies without becoming an always-on cost. Early warnings are best treated as intervention opportunities rather than automatic stop rules because some warned runs recover. A production system could change retrieval strategy, require stronger evidence, reduce budget for low-probability continuations, or route only ambiguous traces to a judge. This is why final-answer accuracy alone is too coarse: no-final traces, weakly grounded answers, and correct but redundant traces have different operational meanings.

A. Limitations

This study uses one agent configuration and one GAIA validation snapshot, so it does not establish generality across models, prompts, or benchmarks. The 10-trace grounding evaluation and 30-trace correctness evaluation are descriptive. Token and tool-call counts are cost proxies rather than direct energy, latency, or billing measurements. Embedding similarity can miss entailment and numerical correctness, and the intervention pilot is too small to estimate accuracy-cost tradeoffs.

B. Conclusion

This paper introduced failure-aware observability for diagnosing wasted computation in tool-using multi-agent LLM systems. On 165 GAIA validation traces, unusable outcomes arise from evidence gaps, repeated loops, tool issues, and step limits rather than one failure mode. Cheap metrics identify wasteful trajectories early, semantic metrics deepen grounding analysis, and selective LLM-as-judge evaluation validates the analysis without becoming always-on. The next step is to measure controlled intervention policies for cost-adjusted task success.

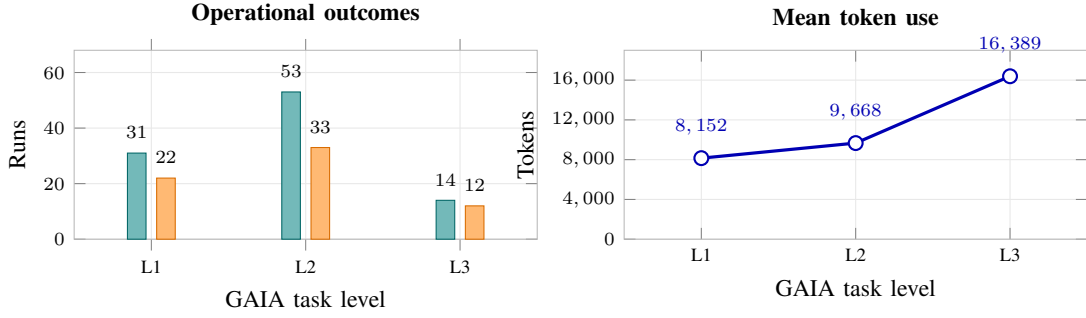


Fig. 2. Full level-stratified run under identical run caps. Usable finals are operational outcomes, not correctness claims: 31/53, 53/86, and 14/26 runs are usable at levels 1–3, with mean token use of 8,152, 9,668, and 16,389. Teal bars denote usable finals; orange bars denote failed/no-final runs.

TABLE II
FULL LEVEL-STRATIFIED METRICS. SUPPORT IS THE ANSWER-SENTENCE FRACTION ABOVE THE 0.65 EVIDENCE-SIMILARITY THRESHOLD.

Level	Runs	Usable	Failed	Triage	Evidence	Tool err.	Tokens	Tools	Support
1	53	31	22	23	0.811	0.189	8,152.0	3.02	0.047
2	86	53	33	44	0.977	0.103	9,667.5	3.71	0.201
3	26	14	12	16	0.950	0.109	16,388.8	5.00	0.000

TABLE III
CONCRETE TRACE CASE STUDIES. ACTION ORDER IS PRESERVED AND ACTOR ROLES ARE SHOWN EXPLICITLY; REPEATED ENVIRONMENT OUTPUTS ARE COMPRESSED FOR READABILITY.

Case	Role-labeled trajectory excerpt	Observed signal	Triage implication
Repeated loop (23dd907f, L1)	Orch.: seek Audre Lorde poem indentation → Search: ok. Orch.: repeat same seek → Search: ok. Orch.: inspect Poetry Foundation → Search: ok; inspect same page → ok; inspect Poeticous → ok; inspect same page → ok; inspect Poetry Foundation again → ok. Final status: <code>repeated_action_loop</code> .	LOOPING_WARNING; SEARCH_REPEAT_BLOCKED; 11,749 tokens; near-zero answer-evidence similarity.	The run is active but strategically unchanged; the orchestrator should redirect search or stop before more budget is spent.
Weak grounding (dc22a632, L1)	Orch.: seek Ali Khan / New Mexican restaurant / James Beard winners → Search: ok; repeat and refine seek twice → ok; inspect Civil Alchemy product page → ok; inspect same page → ok. Final status: <code>final_ok</code> .	Evidence present; no tool errors; 9,534 tokens; sentence support near zero; answer-evidence similarity effectively zero.	A clean final answer is not necessarily grounded; semantic evaluation or a judge check should precede acceptance.
Tool recovery (b2c257e0, L2)	Orch.: use task material three times → Search: ok; seek U.S. ice-cream butterfat standards → ok; inspect Wikipedia → ok. Orch.: invoke execution three times → Exec.: errors. Final status: <code>final_ok</code> .	TOOL_FAILURE_STREAK; tool-error rate 0.333; support 0.682; answer-evidence similarity 0.633; 8,869 tokens.	Tool failures should be interpreted with recovery context; strong later grounding can justify continuation instead of immediate stop.
Budget pressure (f5a8cc84, L3)	Orch.: use materials → ok; seek NIH translation / Michaelis-Menten equation → Search: ok; inspect article → ok. Orch.: invoke execution repeatedly → Exec.: ok but no decisive final state. Final status: <code>max_steps_exceeded</code> .	BUDGET_PRESSURE_LOW_IG; LOW_INFO_GAIN_STREAK; EXECUTION_NO_OUTPUT; 27,293 tokens; support 0.037.	On harder tasks, warnings can arrive while substantial budget remains; repeated low-gain computation should trigger redirection or budget gating.

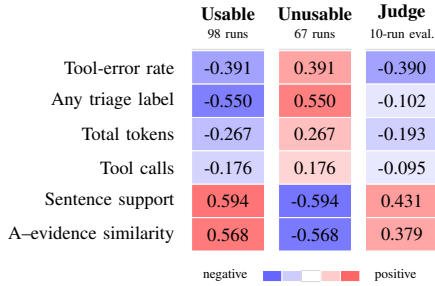


Fig. 3. Pearson r by metric layer. Operational-outcome columns use all 165 runs (98 usable, 67 unusable); judge-support correlations use the cached 10-trace grounding evaluation.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.

- [2] T. Schick, J. Dwivedi-Yu, R. Dessi, et al., “Toolformer: Language models can teach themselves to use tools,” in *Proc. NeurIPS*, 2023.
[3] Q. Wu, G. Bansal, J. Zhang, et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *Proc. COLM*, 2024.
[4] J.-T. Huang, J. Zhou, T. Jin, et al., “On the resilience of LLM-based multi-agent collaboration with faulty agents,” in *Proc. ICML*, PMLR, vol. 267, pp. 26202–26226, 2025.
[5] M. Cemri, M. Z. Pan, S. Yang, et al., “Why do multi-agent LLM systems fail?” in *Proc. NeurIPS*, 2025.
[6] G. Mialon, C. Fourrier, C. Swift, et al., “GAIA: A benchmark for general AI assistants,” in *Proc. ICLR*, 2024.
[7] Y. Du, J. Feng, J. Zhao, and Y. Li, “TrajAgent,” in *Proc. NeurIPS*, 2025.
[8] B. H. Sigelman, L. A. Barroso, M. Burrows, et al., “Dapper,” Google, Tech. Rep., 2010.
[9] N. Reimers and I. Gurevych, “Sentence-BERT,” in *Proc. EMNLP*, 2019.
[10] “Demystifying evals for AI agents,” <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>, accessed 7 June 2026.
[11] “Introducing GPT-4.1 in the API,” <https://openai.com/index/gpt-4-1/>, accessed 7 June 2026.
[12] F. Oviedo, F. Kazhamiaka, E. Choukse, et al., “Energy use of AI inference, efficiency pathways, and test-time scaling,” *Joule*, p. 102430, 2026, doi: 10.1016/j.joule.2026.102430.
[13] M. Yuan, J. Liu, J. Yang, et al., “TA-Mem,” in *Proc. ICAACE*, 2026, pp. 2684–2688.
[14] P. Liang, M. Yuan, J. Liu, et al., “DynaRAG,” in *Proc. ISBDAS*, 2026, pp. 442–445.
[15] W. Yan, Y. Wu, P. Liang, et al., “PRISM,” in *Proc. GAIS*, 2026, pp. 709–712.
[16] Z. Li, C. Ren, X. Li, et al., “Learning skill equivalencies across platform taxonomies,” *LAK*, 2021.